

17 — Petits exos flash de révision

25 mini-exercices rapides (1 à 2 min chacun) pour t'entraîner AVANT la soutenance. Format : une question → tu réponds à voix haute → tu vérifies avec « 💡 Réponse ». But : savoir **retrouver** le code et **l'expliquer** simplement, pas le réciter.

Partie A — « Où est le code qui... ? » (repérage)

A1

Où est calculée la commission de 5 % sur une vente ? 💡 `API/admin/stripe.go`, fonction `PaymentAnnonce` : `commission := (unitAmount * 5) / 100`.

A2

Dans quel fichier on découpe les annonces par pages de 24 ? 💡

`Frontend/script/annonces/annonceAll.js`, fonction `renderPageAnnonces`, ligne `items.slice(debut, debut + ANN_PAR_PAGE)`.

A3

Où sont approuvés / masqués les messages du forum (backend) ? 💡 `API/bdd/forumReq.go`, fonction `ModerateForumMessage` (`switch action` → `approuver` / `masquer`).

A4

Quel fichier adapte la barre de navigation selon le rôle (Pro/Client) ? 💡

`Frontend/script/navRole.js` : ajoute `theme-pro` et réécrit les liens `/client` vers `/pro`, `/admin`, `/salarie`.

A5

Où est stocké le rôle de l'utilisateur après connexion ? 💡 `Frontend/script/login.js` : `localStorage.setItem("userRole", donneesServeur.role)`.

A6

Quel fichier reçoit la confirmation de paiement de Stripe ? 💡 `API/admin/webhook.go`

(`StripeWebhookHandler`), route `POST /api/stripe/webhook` .

A7

Où sont envoyées les notifications (base + push) ? 💡 `API/admin/notifications.go` , fonction `SendPushNotification` (insère en base + appelle OneSignal).

A8

Comment on protège une route pour qu'elle soit réservée aux Admins ? 💡 Dans `API/route/...` : `auth.VerifyRoleMiddleware(handler, "Administrateur")` .

Partie B — « Que fait cette ligne ? » (lecture de code)

B1

```
const nbPages = Math.max(1, Math.ceil(items.length / ANN_PAR_PAGE));
```

💡 Calcule le nombre total de pages, arrondi **vers le haut** (`Math.ceil`), avec **au moins 1** page.

B2

```
unitAmount := int64(prix * 100)
```

💡 Convertit le prix en **centimes** (Stripe raisonne en centimes) : 25 € → 2500.

B3

```
Authorization: "Bearer " + monToken
```

💡 Envoie le **jeton JWT** dans l'en-tête pour prouver qu'on est connecté (sinon le backend répond 401).

B4

```
WHERE m.est_moderne = 0
```

💡 Ne récupère que les messages **non masqués** (masquer = mettre `est_modere = 1`).

B5

```
token, err := auth.GenerateJWT(userBdd.Id, userBdd.Role)
```

💡 Crée le **jeton de connexion** signé, contenant l'id et le rôle de l'utilisateur.

B6

```
window.location.href = data.url;
```

💡 Redirige le navigateur vers la **page de paiement Stripe** renvoyée par le backend.

B7

```
if (!localStorage.getItem("token")) { window.location.replace("/login"); }
```

💡 Si l'utilisateur n'a pas de jeton (pas connecté), on le renvoie vers la page de **connexion**.

Partie C — « Le jury te demande... » (réflexe oral)

C1

« **Montrez-moi que le paiement fonctionne.** » 💡 Se connecter, ouvrir une annonce, « Acheter », payer avec la carte de test `4242 4242 4242 4242` (date future, CVC au hasard), montrer le retour `payment=success` .

C2

« **Comment gérez-vous les droits d'accès ?** » 💡 JWT à la connexion → le token contient le rôle → `VerifyTokenMiddleware` (connecté ?) et `VerifyRoleMiddleware` (bon rôle ?) protègent les routes. Côté front, `auth_guard.js` (`checkSession`) redirige si le rôle ne correspond pas.

C3

« **Que se passe-t-il si vous avez 50 000 annonces ?** » 💡 La pagination côté client montrerait ses limites (on télécharge tout) → il faudrait passer à une pagination **côté serveur** (`LIMIT` / `OFFSET`). En attendant, on a nettoyé la base + paginé l'affichage.

C4

« **Où stockez-vous les mots de passe ?** » 💡 Jamais en clair : hachés avec **bcrypt** avant l' `INSERT` .
À la connexion on compare le hash, on ne déchiffre rien.

C5

« **Comment un vendeur reçoit-il son argent ?** » 💡 Via **Stripe Connect** : chaque vendeur a un compte Stripe relié. Stripe verse le montant au vendeur et prélève automatiquement notre commission de 5 % (`ApplicationFeeAmount` + `TransferData`).

C6

« **Comment les messages du forum s'affichent en temps réel ?** » 💡 Par **polling** : `setInterval` recharge les messages toutes les 4 secondes (`Frontend/script/forum.js`), sans que l'utilisateur rafraîchisse.

C7

« **Faites une modif en direct : commission à 10 %.** » 💡 `API/admin/stripe.go` → remplacer `* 5`) / `100` par `* 10`) / `100` (et `0.05` → `0.10`), puis `cd API && go build ./...` , rebuild du conteneur.

Partie D — Mini-défis chronométrés (🕒 2-3 min)

D1 — Changer le nombre d'annonces par page

💡 `annonceAll.js` : `const ANN_PAR_PAGE = 24;` → mettre `12` . Recharger `/annonces` .

D2 — Ajouter un `console.log` du nombre de pages

💡 Dans `renderPageAnnonces` , après le calcul : `console.log("Pages :", nbPages);` . Vérifier dans F12 → Console.

D3 — Masquer un message du forum directement en SQL

💡 `UPDATE message_forum SET est_moderé = 1 WHERE id_message = 3;` → il disparaît de la modération.

D4 — Trouver combien d'annonces sont sponsorisées

💡 `SELECT COUNT(*) FROM annonce WHERE is_sponsored = 1;`

Plan d'entraînement (30 min la veille)

1. **Partie A** en entier (repérage) — c'est ce qui rassure le plus le jury : tu sais où est ton code.
2. **C1, C2, C5** (paiement, droits, Stripe) — les questions les plus probables.
3. **D1 + C7** (une modif front + une modif back en direct) — pour montrer que tu maîtrises.
4. Relis les fiches **15** (pagination) et **16** (Stripe) pour les détails.